
Desarrollo de Aplicaciones I Trabajo Práctico Obligatorio

OBJETIVO GENERAL DEL PROYECTO

El propósito de este TPO es el diseño e implementación integral de una aplicación móvil nativa bajo la plataforma Android, utilizando Kotlin como lenguaje núcleo. Se exige un desarrollo alineado con los estándares de la industria actual, priorizando la escalabilidad, la robustez arquitectónica y la optimización de recursos. El estudiante deberá proponer un proyecto original que, tras ser validado por el cuerpo docente, sirva como vehículo para demostrar dominio técnico en arquitecturas distribuidas y modernas interfaces declarativas.

STACK TECNOLÓGICO Y REQUISITOS ARQUITECTÓNICOS

Para alcanzar la aprobación, la solución técnica debe integrarse obligatoriamente bajo los siguientes pilares de ingeniería:

- **Lenguaje:** Kotlin
- **Interfaz de Usuario:** Uso avanzado de Jetpack Compose. Se exige la implementación de State Hoisting y el manejo de estados mediante *collectAsStateWithLifecycle* para garantizar la eficiencia en el ciclo de vida de los componentes.
- **Arquitectura:** Patrón MVVM (Model-View-ViewModel) con una separación de capas estricta e infranqueable. Es obligatorio el uso del Repository Pattern para mediar entre las fuentes de datos.
- **Inyección de Dependencias:** Implementación obligatoria una librería de gestión de dependencias para la gestión del grafo de dependencias y desacoplamiento de componentes.
- **Estrategia de Datos "Offline-first":** El flujo de datos debe garantizar que la Base de Datos Local (Room) sea la Única Fuente de Verdad (Single Source of Truth) para la UI. La aplicación debe observar Room; Retrofit se encargará exclusivamente de actualizar la persistencia local tras el fetching de la API.

FUNCIONALIDADES MÍNIMAS REQUERIDAS

Pantalla de Splash

Transición fluida mediante la API de Splash de Android, asegurando una carga de branding inicial profesional.

Autenticación con Persistencia de Sesión

Integración de Registro y Login vía Google Firebase (específicamente cuentas Gmail). El estado de la sesión debe persistir entre reinicios de la aplicación, redirigiendo al usuario al flujo principal sin necesidad de re-autenticación.

Listado de Datos Dinámico

Implementación de una pantalla con LazyColumn que recupere información de una API REST externa (Retrofit).

Requisitos:

Manejo de Estados: La UI debe reflejar de forma explícita los estados de Loading, Success y Error.

Búsqueda y Filtrado: Sistema de búsqueda reactivo sobre el listado.

Detalle del Elemento

Navegación hacia una vista profunda de datos.

Renderizado de imágenes

El sistema debe implementar alguna funcionalidad que recupere URLs de imágenes y las cargue dinámicamente mediante el uso de la librería Glide, aplicando transformaciones o placeholders según el estado de la carga.

Persistencia y Sincronización Remota

Room: Persistencia local obligatoria para el funcionamiento offline de los datos consultados.

Firebase Realtime Database/Firestore: Gestión remota de los datos.

AI SDK

La aplicación deberá contener algún caso de uso que haga uso de un modelo de IA Generativa

DOCUMENTACIÓN DE DISEÑO

La presentación de Wireframes y el prototipo funcional en Figma (o similar) es un

bloqueante: si el maquetado no es aprobado en la primera instancia, el alumno no podrá proceder con las siguientes entregas.

CALIDAD DE SOFTWARE, TESTING Y PERFORMANCE

Testing Unitario y de Integración

No se aceptarán proyectos sin cobertura de tests. Es obligatoria la implementación de pruebas unitarias para:

- ViewModels: Validación de estados de UI y lógica de navegación.
- Use Cases / Repositories: Pruebas de integración simulando respuestas de la API y la DB mediante frameworks de Mocking (ej. MockK o Mockito).

Análisis de Performance (Android Profiler)

El alumno deberá presentar un informe técnico utilizando el Android Profiler, demostrando que no tiene degradaciones de performance por recomposiciones en Compose o memory leaks.

GESTIÓN DEL CÓDIGO

Uso mandatorio de Git bajo estrategias de branching: cada funcionalidad debe poseer su propia rama antes del merge a la rama principal. Se evaluará el proceso y no el resultado final, por lo cual **no se considerarán proyectos que solo tengan un commit con el resultado final.**

ENTREGABLES

Entrega	Contenidos	Fecha
Primera	Documento de definición, Casos de Uso, Endpoints API, Diccionario de Datos y Maquetado en Figma	22/04/2026
Segunda	Implementación de Splash, Auth Firebase, Listado con búsquedas y pantalla de Detalle. Entregables de la primer entrega	27/05/2026
Final	Implementación de Room (offline-first), Testing Unitario, Firebase y Profiler Report. Entregables de la primer y segunda entrega	24/06/2026

CRITERIOS DE EVALUACIÓN Y DEFENSA

La evaluación final se regirá por la **solidez técnica** y la **autoría comprobada**:

1. Arquitectura y Código: Aplicación correcta de MVVM, Repository Pattern y legibilidad profesional.
2. Robustez: Manejo de excepciones
3. Defensa Individual: Durante el examen final, el docente podrá solicitar explicaciones sobre el funcionamiento de algún código, que se indique que código resuelve alguna funcionalidad, modificaciones de código o refactorizaciones en tiempo real.
4. Honestidad Académica: El plagio o la falta de conocimiento sobre el código entregado se sancionará bajo el régimen disciplinario de la Facultad.

ESTRUCTURA DEL DOCUMENTO DE DEFINICION

El documento debe seguir este esquema para su validación:

1. Nombre y Objetivo: Propuesta de valor de la Aplicación
2. Casos de Uso: Definición de la interacción del usuario. Se debe explicar de manera detallada el funcionamiento de cada una de las pantallas. Si la pantalla va a consumir servicios externos, ya sea endpoints de alguna API o base de datos, se deberá detallar que datos se espera enviar o guardar o que datos se espera recibir o recuperar.
3. Diccionario de Datos: Esquema de tablas en Room y colecciones en Firebase.
4. Especificación de Endpoints: Detalle de Verbo HTTP, URL, Headers y estructura JSON (Request/Response), indicando pantalla donde se va a utilizar.
5. Enlace a Figma (o similar): Acceso al prototipo funcional.
6. Enlace a GIT publico